

Instructor: Date:

Learner: Regional Branch:

Program: Center:

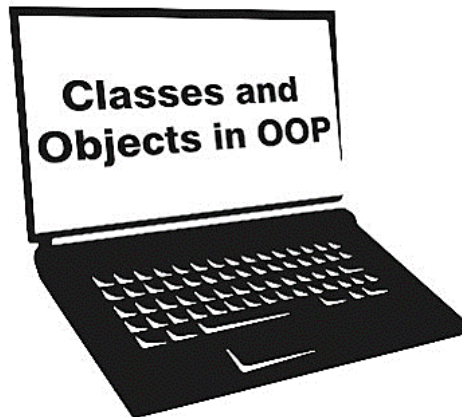
CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Identifying Classes and Objects in OOP.



Learning Outcome:

Describe common activities at work to a using short sentences in familiar spoken and written situations.

Make the result report from the analysis of the information system, based on the customer requirements, following established rules and protocols.

Can Do Statements:

- I can explain briefly what classes and objects are in OOP.
- I can give short examples about classes and objects in OOP.
- I can identify classes and objects in OOP.

Ask your partner:

- Is there any difference between classes and objects in OOP? Which one?
- How can you identify classes and objects in OOP?





Let's Explore!

A Brochure* with OOP Information.

*A "brochure" is a booklet in which relevant information of a product or a company is written.



Instructions: Read the text below.

Charlie: I came across this brochure about OOP. There was a bunch of these displayed in the main hall of the faculty.

Mark: What does it say?

Charlie: It says that the big difference between OOP and procedure programming languages is that instead of writing a long piece of code, people who program with OOP decided to divide long ago information in objects, which they defined as encapsulated data.

Mark: I know that there are two key concepts for understanding OOP, which are objects and classes. The brochure must portray this.

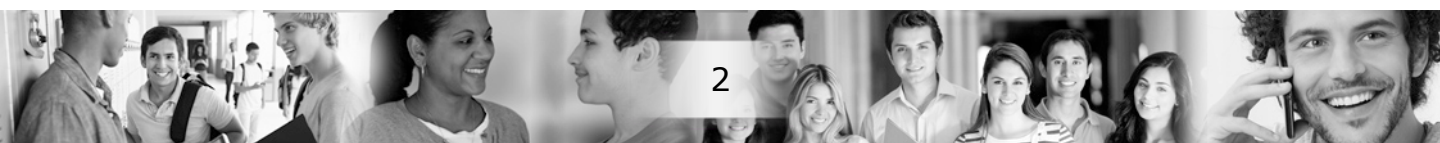
Charlie: Indeed. For stating the difference between those concepts, the authors use the example of a washing machine. They define class as a general category in which some other elements can be contained. For instance, "machines" would be the class for the washing machine. Now, what does the class define? It refers to two sets of features: properties and methods. Properties are the variables or data and methods, the functions.

Mark: Let me guess. When talking about the washing machine, properties must be all the features related to height, weight, price and brand. Regarding methods, we can talk about concrete actions such as washing, rinsing and drying.

Charlie: Exactly. According to the brochure, a programmer made all the description as an abstract idea, a blueprint of something through certain class in OOP, while the object is the concrete idea, the representation of the data and a program execution. The brochure reminds us that designers built each object from a class and each class is designed and programmed to accomplish various tasks.

Mark: What's that information at the end of the brochure about?

Charlie: It says that many apps, such as Textfield and Array, are using this OOP.



**Let's Practice!**

Classes and Objects in OOP? Is this difference clear?

Instructions: Select the correct answer.

1. In OOP, what is a class?
 - a. A general category
 - b. One isolated category
 - c. Specific categories of an element
 - d. Properties of an object
2. What does the class define?
 - a. The objects and the properties
 - b. The encapsulated data
 - c. Properties and methods
 - d. Object categories
3. What is a property in OOP?
 - a. Methods
 - b. Variables or data
 - c. Functions
 - d. A set of features

**Let's Practice!**

Examples of classes and objects in OOP.

Instructions: Match column A with column B. Read the definitions aloud. Check your pronunciation with your instructor.

COLUMN A

1. In programming language, experts classified features such as height and weight as
2. Experts call a certain function in OOP:
3. Experts define a broad category as "machines" as a
4. In OOP, a class is a
5. Programmers who use OOP divide information in
6. Taking into account OOP, an object is a

COLUMN B

- a. properties.
- b. method.
- c. class in OOP.
- d. general category.
- e. objects.
- f. concrete idea.





Extension Activity

Let's go over objects and classes in OOP.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Look for information about examples of classes and objects in OOP.
- Describe how they work.
- Write a summary of the information.
- Show your summary to your classmates and teacher and talk about it.

Self-Assessment

- | | Yes | No |
|---|-----------------------|-----------------------|
| 1. I look for information about examples of classes and objects in OOP. | ☐ | ☐ |
| 2. I describe how they work. | ☐ | ☐ |
| 3. I write a summary related to the topic. | ☐ | ☐ |
| 4. I share my summary and talk about it. | ☐ | ☐ |



Target Words

- **blueprint** (noun): it can be either a plan that explains how to achieve something or the drawing in which architects show how to build a structure
e.g. According to the brochure, a programmer made all the description as an abstract idea, a blueprint of something through certain class in OOP.
- **array** (noun): a data map that links keys and values to create objects
e.g. It says that many apps, such as Textfield and Array, used this OOP.
- **come across** (phrasal verb): to find something or someone by chance
e.g. I came across this brochure about OOP.
- **displayed** (adjective): placed somewhere to be easily seen
e.g. There was a bunch of these displayed in the main hall of the faculty.
- **portray** (verb): to represent or describe someone or something through a painting, film, book, or other type of work
e.g. The brochure must portray this.
- **execution** (noun): the act of doing something, especially in a planned way
e.g. The object is the concrete idea, the representation of the data and a program execution.



Learning Points

Action Verbs Used in Programming in Present, Past and Continuous Tenses.

Here are some of the many verbs you can use when describing programming languages. You can use these verbs to describe how OOP works.

In this chart, you will find the basic forms of verbs in English language that will help you describe programming languages like OOP. It is very important to highlight that verbs will change depending on the grammar structure that the specific tense requires.

Base Form	Present Tense*	Past Tense**	Past Participle**	-ing Form
To design	Design/	Designed	Designed	Designing
To define	designs	Defined	Defined	Defining
To characterize	Define/	Characteriz	Characterized	Characterizi
To fill	defines	ed	Filled	ng
To create		Filled	Created	Filling
	Characteri	Created		Creating
	ze/			
	characteriz			
	es			
	Fill/fills			
	Create/			
	creates			

*Remember that for constructing a sentence in simple present, it is vital to be careful when conjugating the verb in third person singular since there is a variation for this particular case (you add and -s or an -es at the end).

**When using past tense in English, we have to take into account that there are *irregular* and *regular* verbs and their structure is different. In this case, the chart only shows how regular verbs work (the suffix -ed is added).

Example

- The brochure says that many apps, such as Textfield and Array, are using this OOP. People who program with OOP decided long ago to divide information in objects, which they defined as encapsulated data.



Learning Points

Modal Verb Must.

The modal verb **must** has different uses. The most common use refers to expressing that something is mandatory, like an obligation or duty. You can use this grammatical construction when talking about the elements or essential features that a programming paradigm as OOP has.

Take a look at the following grammatical structure:

Subject + **must** + verb in present tense + complement

Example:

You **must** know how classes and objects work in OOP.

The following chart shows other uses for the modal verb **must**:

Use	Example
Obligation	<i>The students must take the exam before graduating.</i>
Strong recommendation	<i>You must stop smoking; it's for your own health.</i>
Deduction	<i>Lauren is pale, she must be sick.</i>

Examples

- *In programming terminology, an object is a component that must contain properties and methods needed to make data useful.*
- *When talking about the washing machine, properties must be all the features related to height, weight, price and brand.*